arXiv:2309.08600v3 [cs.LG] 4 Oct 2023

SPARSE AUTOENCODERS FIND HIGHLY INTERPRETABLE FEATURES IN LANGUAGE MODELS

Hoagy Cunningham[*12], **Aidan Ewart**[*13], **Logan Riggs**[*1], **Robert Huben**, **Lee Sharkey**[4]
[1]EleutherAI, [2]MATS, [3]Bristol AI Safety Centre, [4]Apollo Research
{hoagycunningham, aidanprattewart, logansmith5}@gmail.com

ABSTRACT

One of the roadblocks to a better understanding of neural networks' internals is *polysemanticity*, where neurons appear to activate in multiple, semantically distinct contexts. Polysemanticity prevents us from identifying concise, human-understandable explanations for what neural networks are doing internally. One hypothesised cause of polysemanticity is *superposition*, where neural networks represent more features than they have neurons by assigning features to an overcomplete set of directions in activation space, rather than to individual neurons. Here, we attempt to identify those directions, using sparse autoencoders to reconstruct the internal activations of a language model. These autoencoders learn sets of sparsely activating features that are more interpretable and monosemantic than directions identified by alternative approaches, where interpretability is measured by automated methods. Moreover, we show that with our learned set of features, we can pinpoint the features that are causally responsible for counterfactual behaviour on the indirect object identification task (Wang et al., 2022) to a finer degree than previous decompositions. This work indicates that it is possible to resolve superposition in language models using a scalable, unsupervised method. Our method may serve as a foundation for future mechanistic interpretability work, which we hope will enable greater model transparency and steerability.

1 INTRODUCTION

Advances in artificial intelligence (AI) have resulted in the development of highly capable AI systems that make decisions for reasons we do not understand. This has caused concern that AI systems that we cannot trust are being widely deployed in the economy and in our lives, introducing a number of novel risks (Hendrycks et al., 2023), including potential future risks that AIs might deceive humans in order to accomplish undesirable goals (Ngo et al., 2022). Mechanistic interpretability seeks to mitigate such risks through understanding how neural networks calculate their outputs, allowing us to reverse engineer parts of their internal processes and make targeted changes to them (Cammarata et al., 2021; Wang et al., 2022; Elhage et al., 2021).

To reverse engineer a neural network, it is necessary to break it down into smaller units (features) that can be analysed in isolation. Using individual neurons as these units has had some success (Olah et al., 2020; Bills et al., 2023), but a key challenge has been that neurons are often *polysemantic*, activating for several unrelated types of feature (Olah et al., 2020). Also, for some types of network activations, such as the residual stream of a transformer, there is little reason to expect features to align with the neuron basis (Elhage et al., 2023).

Elhage et al. (2022b) investigate why polysemanticity might arise and hypothesise that it may result from models learning more distinct features than there are dimensions in the layer. They call this phenomenon *superposition*. Since a vector space can only have as many orthogonal vectors as it has dimensions, this means the network would learn an overcomplete basis of non-orthogonal features. Features must be sufficiently sparsely activating for superposition to arise because, without

[*]Equal contribution
Code to replicate experiments can be found at https://github.com/HoagyC/sparse_coding

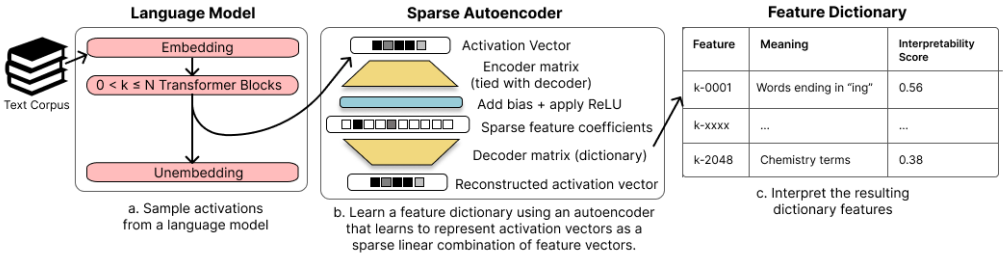

Figure 1: An overview of our method. We a) sample the internal activations of a language model, either the residual stream, MLP sublayer, or attention head sublayer; b) use these activations to train a neural network, a sparse autoencoder whose weights form a feature dictionary; c) interpret the resulting features with techniques such as OpenAI's autointerpretability scores.

high sparsity, interference between non-orthogonal features prevents any performance gain from superposition. This suggests that we may be able to recover the network's features by finding a set of directions in activation space such that each activation vector can be reconstructed from a sparse linear combinations of these directions. This is equivalent to the well-known problem of sparse dictionary learning (Olshausen & Field, 1997).

Building on Sharkey et al. (2023), we train sparse autoencoders to learn these sets of directions. Our approach is also similar to Yun et al. (2021), who apply sparse dictionary learning to all residual stream layers in a language model simultaneously. Our method is summarised in Figure 1 and described in Section 2.

We then use several techniques to verify that our learned features represent a semantically meaningful decomposition of the activation space. First, we show that our features are on average more interpretable than neurons and other matrix decomposition techniques, as measured by autointerpretability scores (Section 3) (Bills et al., 2023). Next, we show that we are able to pinpoint the features used for a set task more precisely than other methods (Section 4). Finally, we run case studies on a small number of features, showing that they are not only monosemantic but also have predictable effects on the model outputs, and can be used for fine-grained circuit detection. (Section 5).

2 Taking Features out of Superposition with Sparse Dictionary Learning

To take network features out of superposition, we employ techniques from *sparse dictionary learning* (Olshausen & Field, 1997; Lee et al., 2006). Suppose that each of a given set of vectors $\{\mathbf{x}_i\}_{i=1}^{n_{\text{vec}}} \subset \mathbb{R}^d$ is composed of a sparse linear combination of unknown vectors $\{\mathbf{g}_j\}_{j=1}^{n_{\text{gt}}} \subset \mathbb{R}^d$, i.e. $\mathbf{x}_i = \sum_j a_{i,j}\mathbf{g}_j$ where $\mathbf{a_i}$ is a sparse vector. In our case, the data vectors $\{\mathbf{x}_i\}_{i=1}^{n_{\text{vec}}}$ are internal activations of a language model, such as Pythia-70M (Biderman et al., 2023), and $\{\mathbf{g}_j\}_{j=1}^{n_{\text{gt}}}$ are unknown, ground truth network features. We would like learn a dictionary of vectors, called dictionary features, $\{\mathbf{f}_k\}_{k=1}^{n_{\text{feat}}} \subset \mathbb{R}^d$ where for any network feature $\mathbf{g}_j$ there exists a dictionary feature $\mathbf{f}_k$ such that $\mathbf{g}_j \approx \mathbf{f}_k$.

To learn the dictionary, we train an autoencoder with a sparsity penalty term on its hidden activations. The autoencoder is a neural network with a single hidden layer of size $d_{\text{hid}} = Rd_{\text{in}}$, where d_{in} is the dimension of the language model internal activation vectors[1], and R is a hyperparameter that controls the ratio of the feature dictionary size to the model dimension. We use the ReLU activation function in the hidden layer (Fukushima, 1975). We also use tied weights for our neural network, meaning the weight matrices of the encoder and decoder are transposes of each other.[2] Thus, on

[1]We mainly study residual streams in Pythia-70M and Pythia 410-M, for which the residual streams are of size $d_{\text{in}} = 512$ and $d_{\text{in}} = 1024$, respectively (Biderman et al., 2023)

[2]We use tied weights because (a) they encode our expectation that the directions which detect and define the feature should be the same or highly similar, (b) they halve the memory cost of the model, and (c) they remove

input vector $\mathbf{x} \in \{\mathbf{x}_i\}$, our network produces the output $\hat{\mathbf{x}}$, given by

$$\mathbf{c} = \mathrm{ReLU}(M\mathbf{x} + \mathbf{b}) \tag{1}$$

$$\hat{\mathbf{x}} = M^T\mathbf{c} \tag{2}$$

$$= \sum_{i=0}^{d_{\text{hid}}-1} c_i \mathbf{f}_i \tag{3}$$

where $M \in \mathbb{R}^{d_{\text{hid}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{hid}}}$ are our learned parameters, and M is normalised row-wise[3]. Our parameter matrix M is our feature dictionary, consisting of d_{hid} rows of dictionary features $\mathbf{f}_i$. The output $\hat{\mathbf{x}}$ is meant to be a reconstruction of the original vector $\mathbf{x}$, and the hidden layer $\mathbf{c}$ consists of the coefficients we use in our reconstruction of $\mathbf{x}$.

Our autoencoder is trained to minimise the loss function

$$\mathcal{L}(\mathbf{x}) = \underbrace{||\mathbf{x} - \hat{\mathbf{x}}||_2^2}_{\text{Reconstruction loss}} + \underbrace{\alpha||\mathbf{c}||_1}_{\text{Sparsity loss}} \tag{4}$$

where α is a hyperparameter controlling the sparsity of the reconstruction. The ℓ^1 loss term on $\mathbf{c}$ encourages our reconstruction to be a sparse linear combination of the dictionary features. It can be shown empirically (Sharkey et al., 2023) and theoretically (Wright & Ma, 2022) that reconstruction with an ℓ^1 penalty can recover the ground-truth features that generated the data. For the further details of our training process, see Appendix B.

3 Interpreting Dictionary Features

3.1 Interpretability at Scale

Having learned a set of dictionary features, we want to understand whether our learned features display reduced polysemanticity, and are therefore more interpretable. To do this in a scalable manner, we require a metric to measure how interpretable a dictionary feature is. We use the automated approach introduced in Bills et al. (2023) because it scales well to measuring interpretability on the thousands of dictionary features our autoencoders learn. In summary, the autointerpretability procedure takes samples of text where the dictionary feature activates, asks a language model to write a human-readable interpretation of the dictionary feature, and then prompts the language model to use this description to predict the dictionary feature's activation on other samples of text. The correlation between the model's predicted activations and the actual activations is that feature's interpretability score. See Appendix A and Bills et al. (2023) for further details.

We show descriptions and top-and-random scores for five dictionary features from the layer 1 residual stream in Table 1. The features shown are the first five under the (arbitrary) ordering in the dictionary.

3.2 Sparse dictionary features are more interpretable than baselines

We assess our interpretability scores against a variety of alternative methods for finding dictionaries of features in language models. In particular, we compare interpretability scores on our dictionary features to those produced by a) the default basis, b) random directions, c) Principal Component Analysis (PCA), and d) Independent Component Analysis (ICA). For the random directions and for the default basis in the residual stream, we replace negative activations with zeros so that all feature activations are nonnegative [4].

ambiguity about whether the learned direction should be interpreted as the encoder or decoder direction. They do not reduce performance when training on residual stream data but we have observed some reductions in performance when using MLP data.

[3]Normalisation of the rows (dictionary features) prevents the model from reducing the sparsity loss term $||\mathbf{c}||_1$ by increasing the size of the feature vectors in M.

[4]For PCA we use an online estimation approach and run the decomposition on the same quantity of data we used for training the autoencoders. For ICA, due to the slower convergence times, we run on only 2GB of data, approximately 4 million activations for the residual stream and 1m activations for the MLPs.